

Document Scanner

Feature Overview

Field	Value
Feature name	Document Scanner
Status	Phase 1 complete; Phase 2 imaging complete
Phase	Phase 1 (core) + Phase 2 (enhanced imaging)
Priority	High

Description

Browser-based document capture with three ways to add content: (1) scan with the device camera, (2) upload an existing file (PDF or image), or (3) choose an existing photo from the device gallery. Camera and gallery images can be enhanced (rotate, crop, brightness, contrast), combined across multiple pages, and saved as an image or a combined PDF. An uploaded PDF is saved as-is. Designed to work inside an iframe within the GoodBarber mobile app shell.

Requirements

Core (Phase 1)

- ☒ Access device camera via browser APIs (getUserMedia)
- ☒ Capture document image from camera feed
- ☒ Basic image enhancement (contrast, brightness adjustment)
- ☐ Edge detection (stretch goal for Phase 1 - not implemented)
- ☒ Generate PDF or image output from the captured scan
- ☒ Save scanned document to cloud storage (S3) with metadata stored in the database
- ☐ Offline capture and sync (deferred - see Offline Support below)

Enhanced (Phase 2)

- ☒ Multi-page document scanning (combine into a single PDF)
- ☒ Interactive crop (draggable rectangle with corner handles)
- ☒ Rotate in 90-degree steps
- ☒ Brightness and contrast sliders (live preview)
- ☐ Automatic perspective correction (not implemented; crop is manual)
- ☐ Document type detection (not implemented)
- ☐ OCR text extraction (stretch goal - not implemented)

Implemented Behaviour (Phase 2)

- There are three ways in. The centre scan button routes to `/scan?pick=camera` and opens straight into the camera with no intermediate screen. The Upload item in the bottom bar does not open the scanner at all: it owns a hidden image-only file input, so tapping it opens the phone's native picker on top of the current screen. "Scan into this vault" from inside a vault opens `/scan` with no pick param and still shows the two-option chooser ("Scan with camera" and "Upload a file or photo"), which is the only path that accepts a PDF.
- Upload handoff: a File cannot travel in a URL, so a photo picked from the bottom-bar Upload input is parked in a small in-memory module (`lib/pending-upload.ts`, single slot, cleared on read) and the app routes to `/scan?pick=pending`, which takes the file and goes straight to the review and edit screen. Cancelling the picker does nothing and leaves the user where they were.
- Accept lists matter on Android. Chrome and Samsung Internet map the accept list onto an Android intent, and the photo gallery picker is only used when the list is image types only. A non-image type such as `application/pdf`, or a bare extension the system cannot resolve to a MIME type (`.heic` and `.heif` are the usual offenders), makes the browser fall back to the generic Camera / Camcorder / Files chooser with no gallery. The Upload path and the in-camera gallery shortcut therefore use a plain `accept="image/*"` with no extensions. Only the in-vault upload input carries `application/pdf`, because that path is meant to reach the file browser.
- Upload a file (in-vault path) accepts a PDF or an image. An image goes into the normal edit flow. A PDF skips the imaging tools and is shown as a "PDF ready to save" review, then uploaded as-is (no re-encoding).
- Cross-platform device handling (iOS and Android): iPhone HEIC/HEIF photos are decoded to JPEG in the browser (`heic2any`) before editing, so they display and save correctly on every platform. This still works with the image-only accept list, because the operating system reports HEIC as an image type and the conversion runs after selection. File type is detected from both the reported MIME type and the filename extension, because Android file managers and content-URI pickers often report an empty or generic type. A "Preparing your photo" overlay is shown while a large photo is decoded.
- The camera view has exactly three controls: Close scanner (top left), Choose from gallery (bottom left) and Capture (centre). Close is a single-step exit: it stops the camera stream and replaces the route with `/home`, so one tap returns to the vaults. There is deliberately no second close control. The no-camera fallback offers upload, a camera retry and a way back to the chooser.
- The scanner has staged flow: chooser, loading, camera, no-camera fallback, edit, saving.
- Edit stage supports rotate left/right, interactive crop with corner handles, and brightness/contrast sliders. Applying a crop flattens the current rotation, filters and crop into a fresh working image.
- Multi-page: each captured page is baked into a page list shown as a thumbnail strip. Pages can be removed, and "Add page" returns to the camera.
- Save output: a single page saves as a JPEG (`mime image/jpeg`). Multiple pages, or the "Save as PDF" toggle, save as one PDF (`mime application/pdf`) built with jsPDF (A4, one image per page, fit with margin).
- Upload path is unchanged in shape: request a presigned URL (now also sending `fileSize` for the quota check), PUT the blob with a matching Content-Type, then POST metadata to `/api/documents`.
- The file-upload fallback (for devices without a usable camera) is preserved.

Technical Notes

Camera Access

- Uses `navigator.mediaDevices.getUserMedia()` for camera access.
- Requests the rear-facing camera by default on mobile:
`{ video: { facingMode: "environment" } }`.
- Provides a file-upload fallback for devices without a usable camera.

Image Processing

- Canvas API for image manipulation. Rotation is applied to a canvas, then the crop sub-rectangle is taken, then brightness/contrast are applied via the canvas filter.
- Helpers live in `lib/imaging.ts`: `loadImage`, `renderProcessedPage`, `buildPdfFromImages`, plus the `CropRect`, `PageEdit` and `ProcessedPage` types.

HEIC/HEIF Handling (iOS)

- iPhones capture photos as HEIC by default. Browsers cannot decode HEIC on a canvas (and Android/desktop cannot display it), so a selected HEIC/HEIF file is converted to JPEG in the browser with `heic2any` (dynamically imported, so the decoder only loads when it is actually needed) before it enters the edit flow.
- After conversion the rest of the pipeline is unchanged: the JPEG is edited and saved exactly like any other image.

PDF Generation

- Uses jsPDF (client-side) to build a single PDF from the captured page images.
- One image per A4 page, scaled to fit with a margin.

PDF Handling in Viewers

- Because documents can now be application/pdf, the document viewer and the folder grid show a PDF placeholder icon plus an open/download action instead of a broken image. Downloads use the correct file extension (.pdf or .jpg).

Iframe Considerations

- Must work inside an iframe (GoodBarber context).
- Camera permissions must propagate through the iframe: requires the `allow="camera"` attribute on the iframe element.
- Note: the live app opens the vault first-party (breaking out of the embed) so that camera, cookies and login work reliably in the in-app browser. See `build_state.md` Builds 9 and 10.

Offline Support (deferred)

- Offline capture, local queueing and background sync are not implemented in Phase 1 or 2. This remains a design goal for a later phase.

Browser Compatibility

- iOS Safari: highest-risk area for camera in an iframe. The first-party break-out approach mitigates this.
- Android Chrome: primary target.
- Desktop browsers: secondary, for testing and demo.

Open Questions and Risks

Risk	Severity	Notes
iOS iframe camera permissions	Critical	Mitigated by opening the vault first-party from the GoodBarber home. Re-test after each redeploy.
Image quality on lower-end devices	Medium	Camera resolution and processing power vary.
PDF file size	Low	Large images produce large PDFs. Export uses a “fast” compression path.

Dependencies

- Cloud storage setup (S3 bucket, upload API)
- Database schema (documents table with metadata)
- Authentication (user ID for document ownership)